

FASTAPI INTERVIEW GUIDE →



Naresh Edagotti
Follow For More



FastAPI Interview Q/A

1. What is FastAPI, and why is it preferred over Flask or Django for ML/AI services?

FastAPI is a modern Python web framework for building APIs quickly. It's preferred for ML/AI services because it's fast, handles async operations well, and has automatic data validation. Flask is simpler but slower, while Django is heavy for just APIs.

Example: If you're deploying a machine learning model that gets thousands of prediction requests per second, FastAPI can handle those requests faster than Flask because it uses async processing. Django would add unnecessary features like admin panels that ML APIs don't need.

2. How does FastAPI achieve high performance compared to traditional Python frameworks?

FastAPI uses ASGI instead of WSGI, which allows async operations. It also uses Starlette for speed and Pydantic for fast data validation. This combination makes it nearly as fast as Node.js or Go frameworks.

Example: When your ML API calls a database to fetch user data while processing predictions, FastAPI can handle other requests simultaneously instead of waiting. Flask would block and wait, making it slower.

3. What is ASGI, and how is it different from WSGI?

ASGI (Asynchronous Server Gateway Interface) supports async operations, while WSGI (Web Server Gateway Interface) is synchronous and blocks during I/O operations. ASGI handles multiple requests concurrently.

Example: With WSGI (Flask), if a request takes 2 seconds to call an external API, the server waits. With ASGI (FastAPI), the server can handle 100 other requests during those 2 seconds.

4. Explain the role of Uvicorn and Starlette in FastAPI.

Uvicorn is the ASGI server that runs FastAPI applications. Starlette is the lightweight web framework that FastAPI is built on top of, providing routing, requests, and responses functionality.

Example: Think of Starlette as the engine of a car and Uvicorn as the wheels. FastAPI adds the comfortable seats and dashboard. When you run `uvicorn main:app`, Uvicorn serves your FastAPI application.

5. How do you define a basic GET and POST endpoint in FastAPI?

Use `@app.get()` decorator for GET requests and `@app.post()` for POST requests. Define the endpoint path and a function that returns data.

Example:

```
@app.get("/predict")
def get_model_info():
    return {"model": "RandomForest", "version": "1.0"}

@app.post("/predict")
def make_prediction(data: InputData):
    result = model.predict(data)
    return {"prediction": result}
```

6. What is automatic interactive API documentation in FastAPI, and how is it generated?

FastAPI automatically creates interactive documentation at [/docs](#) (Swagger UI) and [/redoc](#). It's generated from your code, type hints, and Pydantic models without extra work.

Example: After creating endpoints, visit <http://localhost:8000/docs>. You'll see all your API endpoints with a "Try it out" button to test them directly in the browser without using Postman.

7. What are `@app.get`, `@app.post`, etc., decorators actually doing internally?

These decorators register your function as a route handler for specific HTTP methods. They tell FastAPI to call your function when someone makes that type of request to that path.

Example: When you write `@app.post("/users")`, FastAPI creates a mapping: "POST request to /users" → "call this function". When a POST request arrives at /users, FastAPI automatically calls your function.

8. How does FastAPI handle request parsing and response serialization?

FastAPI uses Pydantic models to automatically parse JSON requests into Python objects and serialize Python objects back to JSON responses. Type hints tell it what to expect and return.

Example: If you define `def predict(data: PredictionInput)`, FastAPI automatically converts incoming JSON `{"features": [1,2,3]}` into a `PredictionInput` object. Your return value is automatically converted to JSON.

9. What is Pydantic, and why is it tightly integrated with FastAPI?

Pydantic is a data validation library that uses Python type hints. FastAPI uses it to automatically validate request data, provide clear error messages, and generate documentation.

Example: If your model expects `{"age": 25}` but receives `{"age": "twenty"}`, Pydantic automatically catches this error and returns a clear message: "age: value is not a valid integer" without you writing validation code.

10. How does FastAPI validate incoming request data automatically?

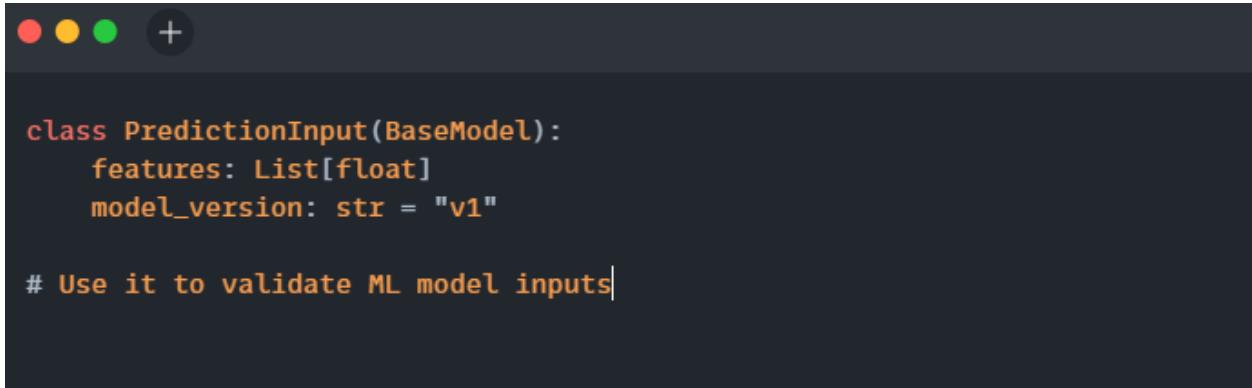
FastAPI checks incoming data against your Pydantic models using type hints. If data doesn't match the expected types or constraints, it automatically returns a 422 error with details.

Example: Define `class User(BaseModel): age: int`. If someone sends `{"age": "abc"}`, FastAPI automatically rejects it and tells the user "age must be an integer" before your function even runs.

11. What is a `BaseModel`, and when would you use it?

`BaseModel` is Pydantic's base class for creating data models. You inherit from it to define request/response schemas with validation, type checking, and serialization.

Example:



```
class PredictionInput(BaseModel):  
    features: List[float]  
    model_version: str = "v1"  
  
# Use it to validate ML model inputs
```

12. How do type hints improve FastAPI's behavior?

Type hints tell FastAPI what data types to expect and return. This enables automatic validation, better error messages, IDE autocomplete, and generates accurate API documentation.

Example: `def predict(age: int, name: str) -> dict:` tells FastAPI to expect an integer and string, validate them, show this in docs, and give autocomplete in your IDE.

13. What happens if the client sends invalid data to a FastAPI endpoint?

FastAPI returns a 422 Unprocessable Entity error with detailed information about what was wrong, including which fields failed validation and why.

Example: Send `{"age": -5}` when age must be positive. FastAPI responds: `{"detail": [{"loc": ["age"], "msg": "ensure this value is greater than 0"}]}`. Your endpoint function never runs.

14. How do you define optional fields, default values, and constraints in Pydantic models?

Use `Optional` from typing for optional fields, assign values for defaults, and use Pydantic's `Field` for constraints like min/max values or string patterns.

Example:

```
class UserInput(BaseModel):
    name: str
    age: Optional[int] = None # Optional
    score: float = Field(ge=0, le=100) # Must be 0-100
```

15. What is the difference between request models and response models?

Request models define what data your endpoint accepts from clients. Response models define what data your endpoint returns. They can be different because input needs might differ from output.

Example: Request: `class CreateUser(BaseModel): name: str, password: str`. Response: `class UserResponse(BaseModel): id: int, name: str` (no password in response for security).

16. What does `async def` mean in FastAPI endpoints?

`async def` makes a function asynchronous, meaning it can pause execution during I/O operations (like database calls) and let other requests run instead of blocking.

Example: `async def get_user(id: int):` can await database queries. While waiting for the database response, FastAPI can process other incoming requests.

17. When should you use `async def` vs normal `def`?

Use `async def` when your function does I/O operations (database, API calls, file reading). Use normal `def` for CPU-heavy work (calculations, ML inference) or when using non-async libraries.

Example: `async def` for fetching user data from MongoDB. Normal `def` for running scikit-learn model predictions since scikit-learn is not async.

18. What is concurrency in FastAPI, and how does it help in handling multiple users?

Concurrency means handling multiple requests at the same time by switching between them during waiting periods. This allows FastAPI to serve many users simultaneously on a single thread.

Example: 10 users request predictions. Each needs to fetch data from a database (1 second each). With concurrency, all 10 complete in ~1 second instead of 10 seconds by interleaving the waiting periods.

19. How does FastAPI handle I/O-bound tasks like database calls or API requests?

FastAPI uses `async/await` to handle I/O-bound tasks. When waiting for I/O, it switches to processing other requests, maximizing efficiency without blocking.

Example: `await db.fetch_user()` pauses this request while fetching from database. FastAPI handles 50 other requests during that pause. When data arrives, it resumes this request.

20. Can FastAPI run CPU-heavy ML inference efficiently? If not, how would you design around it?

No, CPU-heavy inference blocks the event loop. Use background tasks, separate worker processes (`ProcessPoolExecutor`), or dedicated inference services. Consider batching predictions or using async-compatible inference servers.

Example: For heavy deep learning inference, use Celery workers or run inference in a separate process: `loop.run_in_executor(None, model.predict, data)` to avoid blocking FastAPI's main thread.

Scenario-Based Questions

21. You are building a RAG system with FastAPI. How would you structure the API for query input, retrieval, generation, and response?

I'd create a `/rag/query` POST endpoint that accepts user query, retrieves relevant documents from vector DB, generates response using LLM, and returns both answer and sources. Use Pydantic models for input/output validation.

Example:

```
@app.post("/rag/query")
async def rag_query(request: QueryRequest):
    # 1. Validate query
    docs = await vector_db.search(request.query)
    # 2. Retrieve context
    context = format_context(docs)
    # 3. Generate with LLM
    answer = await llm.generate(context, request.query)
    # 4. Return response
    return {"answer": answer, "sources": docs}
```

22. Your FastAPI endpoint is calling a Vector DB and an LLM API. How would you prevent the API from blocking when multiple users hit it at once?

Use `async def` with async database clients and async HTTP clients like `httpx`. This allows FastAPI to handle other requests while waiting for Vector DB and LLM responses. Also implement connection pooling.

Example:

```
async def query_rag(query: str):
    async with httpx.AsyncClient() as client:
        # Non-blocking calls
        docs = await vector_db.search(query)
        response = await client.post(llm_url, json=data)
    return result
```

While one user waits for LLM response (2 seconds), 100 other users can make their Vector DB queries simultaneously.

23. A client sends a huge prompt or document to your API, causing slow responses. How would you handle payload limits and optimization?

Set payload size limits in FastAPI, implement text truncation, use streaming for large documents, and add pagination. Return errors early if content exceeds limits before processing.

Example:

```
from fastapi import Request, HTTPException

@app.post("/analyze")
async def analyze(request: Request):
    body = await request.body()
    if len(body) > 1_000_000: # 1MB limit
        raise HTTPException(413, "Payload too large")

    # Or truncate: text[:5000] for first 5000 chars
```

24. Your ML model takes 8–10 seconds to respond. How would you design the API so the frontend doesn't feel slow?

Implement async processing with background tasks. Return a task ID immediately, let client poll for results or use WebSockets for real-time updates. Store results in Redis/database with the task ID.

Example:

```
@app.post("/predict")
async def predict(data: Input, background_tasks: BackgroundTasks):
    task_id = generate_id()
    background_tasks.add_task(run_model, task_id, data)
    return {"task_id": task_id, "status": "processing"}

@app.get("/result/{task_id}")
async def get_result(task_id: str):
    result = await redis.get(task_id)
    return {"status": "complete", "result": result}
```

25. You need to validate that: a prompt is not empty, temperature is between 0 and 1, top_k is a positive integer. How would you enforce this in FastAPI?

Use Pydantic models with Field constraints. Define min/max values, string lengths, and custom validators. FastAPI automatically validates before the endpoint runs.

Example:

```
from pydantic import BaseModel, Field, validator

class LLMRequest(BaseModel):
    prompt: str = Field(..., min_length=1)
    temperature: float = Field(ge=0, le=1)
    top_k: int = Field(gt=0)

    @validator('prompt')
    def prompt_not_empty(cls, v):
        if not v.strip():
            raise ValueError('Prompt cannot be empty')
        return v
```


26. Your FastAPI service crashes under load during an interview test. What steps would you take to debug and stabilize it?

First, check logs for errors. Add memory profiling, check for connection pool exhaustion, implement rate limiting, add health checks, and use load testing tools to identify bottlenecks. Add proper error handling and timeouts.

Example:

```
# Add connection limits
from fastapi import FastAPI
from slowapi import Limiter
from slowapi.util import get_remote_address

limiter = Limiter(key_func=get_remote_address)
app = FastAPI()
app.state.limiter = limiter

@app.get("/predict")
@limiter.limit("5/minute") # Rate limit
async def predict(request: Request):
    # Add timeout handling
    async with timeout(30):
        return await model.predict()
```

27. How would you add logging and request tracing to a FastAPI-based AI service?

Use Python's logging module with structured logging, add middleware to log all requests/responses, include request IDs for tracing, and log to files or centralized systems like ELK stack.

Example:

```
import logging
import uuid
from fastapi import Request

logging.basicConfig(level=logging.INFO)
logger = logging.getLogger(__name__)

@app.middleware("http")
async def log_requests(request: Request, call_next):
    request_id = str(uuid.uuid4())
    logger.info(f"Request {request_id}: {request.method} {request.url}")
    response = await call_next(request)
    logger.info(f"Response {request_id}: {response.status_code}")
    return response
```

28. If one endpoint fails due to an external API error (LLM down), how would you handle errors gracefully and return meaningful responses?

Use try-except blocks, implement retry logic with exponential backoff, return user-friendly error messages, add circuit breaker patterns, and always include proper HTTP status codes.

Example:

```
from fastapi import HTTPException
import httpx

@app.post("/generate")
async def generate(prompt: str):
    try:
        async with httpx.AsyncClient(timeout=10) as client:
            response = await client.post(llm_api, json={"prompt": prompt})
            return response.json()
    except httpx.TimeoutException:
        raise HTTPException(503, "LLM service is slow, please try again")
    except httpx.RequestError:
        raise HTTPException(503, "LLM service unavailable")
```

29. How would you version your APIs when your AI model or response format changes?

Use URL path versioning ([/v1/predict](#), [/v2/predict](#)) or header-based versioning. Keep old versions running during transition, clearly document changes, and set deprecation timelines.

Example:

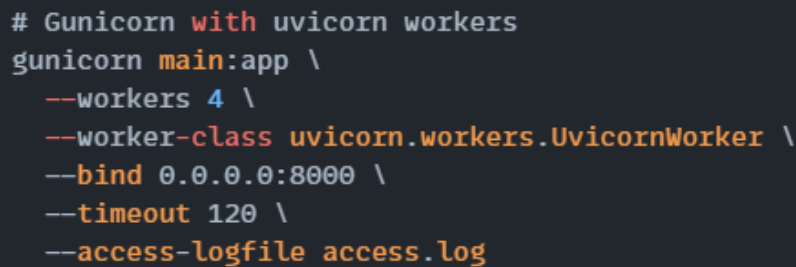
```
# Path versioning
@app.post("/v1/predict")
async def predict_v1(data: InputV1):
    return old_model.predict(data)

@app.post("/v2/predict")
async def predict_v2(data: InputV2):
    # New response format with confidence scores
    result = new_model.predict(data)
    return {"prediction": result, "confidence": 0.95}
```

30. You need to deploy FastAPI for production AI workloads. What would your setup look like (workers, gunicorn, async workers, etc.)?

Use Uvicorn with Gunicorn as process manager, multiple worker processes, set proper worker class (uvicorn.workers.UvicornWorker), add load balancer (Nginx), monitoring, and container orchestration (Docker/Kubernetes).

Example:

A terminal window with a dark background and light-colored text. The window has a title bar with three colored circles (red, yellow, green) and a plus sign. The text inside the terminal is a command to run Gunicorn with uvicorn workers.

```
# Gunicorn with uvicorn workers
gunicorn main:app \
  --workers 4 \
  --worker-class uvicorn.workers.UvicornWorker \
  --bind 0.0.0.0:8000 \
  --timeout 120 \
  --access-logfile access.log
```

Setup: Nginx (load balancer) → Gunicorn (4 workers) → FastAPI app → Redis (caching) → PostgreSQL/Vector DB. Deploy in Docker containers, use Kubernetes for scaling, add Prometheus for monitoring, and implement health checks.